

Agentic Analyst

Enterprise AI SQL Intelligence Platform

Natural Language → SQL → Instant Insights

Document Type:	Full Project Handbook
Version:	2.0
Date:	August 10, 2026
Status:	Active / Production
Author:	Engineering Team

Table of Contents

1	Project Overview	3
2	System Architecture	4
3	Technology Stack	5
4	Project Structure & File Reference	6
5	Backend — Python / FastAPI	7
5.1	API Endpoints Reference	7
5.2	AI Agent Pipeline	9
5.3	Knowledge Graph & Business Logic	10
5.4	SQL Generator & LLM Integration	11
5.5	Prompt Engineering	12
5.6	SQL Validator (AST)	13
5.7	Database Layer	13
5.8	Embedding & Vector Store (ChromaDB)	14
5.9	Query History & Audit Log	14
5.10	Report Generator (CSV, Excel, PDF)	15
6	Frontend — React / Vite	16
6.1	Application Screens	16
6.2	Component Reference	17
7	Environment Configuration (.env)	18
8	Installation & Setup Guide	19
8.1	Prerequisites	19
8.2	Backend Setup	19
8.3	Frontend Setup	20
8.4	Ollama LLM Setup	21
9	Running the Application	22

9.1	Start Backend Server	22
9.2	Start Frontend Dev Server	22
9.3	Build & Serve Frontend (Production)	22
10	API Usage Examples	23
11	Data Flow Walkthrough	24
12	Security & Read-Only Enforcement	25
13	Troubleshooting & FAQ	26
14	Glossary	27

1. Project Overview

Agentic Analyst is a production-grade, enterprise-level AI-powered SQL intelligence platform. It allows non-technical business users to query a live MySQL database using plain English. The system translates natural language questions into valid, optimized SQL queries, executes them in real time, and presents results as rich data tables, auto-generated charts, and downloadable reports — all through a modern conversational web interface.

Core Capabilities

Natural Language Querying	Type business questions in plain English; the system generates the SQL automatically.
AI-Powered SQL Generation	Uses a local Ollama LLM (qwen2.5-coder:7b) with a deterministic fallback synthesizer for offline resilience.
Business Knowledge Graph	An in-memory graph maps business terms, table aliases, column relationships, and entity dictionaries.
Multi-Step Agent Pipeline	Intent detection → Execution planning → SQL generation → AST validation → EXPLAIN check → DB execution → Self-correction loop (up to 3 retries).
Ambiguity Detection	Vague or underspecified queries prompt clarification options backed by real DB enum values.
Read-Only Enforcement	AST-level validation rejects any non-SELECT query — INSERT, UPDATE, DELETE, DROP, etc. are blocked.
Report Export	Results can be exported as CSV, Excel (XLSX), PDF, or JSON directly from the UI.
Auto-Charting	Recharts-powered automatic visualization of query results.
Query History & Audit	Every query is logged with SQL, status, execution time, affected tables, and confidence score.
Admin Dashboard	Schema browser, embedding rebuild, metadata refresh, and system status monitoring.
Settings Panel	Runtime configuration of LLM model, embedding model, temperature, top-k, and row limits.
ChromaDB Vector Store	Semantic search on schema embeddings for RAG-powered schema retrieval.

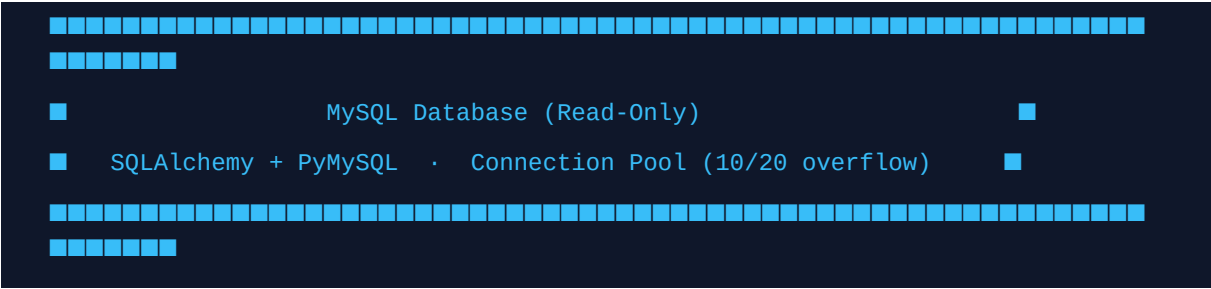
Target Database Domain

The platform is configured for a **JGH enterprise database** containing tables covering users/roles, wallet transactions, SKU inventories, company/brand information, mechanic profiles, withdrawal requests, and automatic bank transactions. All queries are scoped to this defined target table set.

2. System Architecture

The application follows a **clean, layered architecture** with a React frontend, FastAPI backend, and a modular Python AI agent pipeline. Each layer has clear separation of concerns, making it maintainable and extensible.





Communication Flow

Step	Component	Description
1	React Frontend	User types a natural language question in the chat interface
2	HTTP POST /query	Frontend sends question JSON to FastAPI backend
3	Ambiguity Checker	Validates query is not vague; requests clarification if needed
4	Knowledge Graph	Resolves business entities, table aliases, and join paths
5	Query Planner	Builds a deterministic execution plan with table/column/join details
6	Prompt Builder	Constructs a structured LLM prompt from the execution plan
7	SQL Generator	Calls Ollama LLM or deterministic synthesizer fallback
8	SQL Cleaner	Strips markdown artifacts, truncates at first semicolon
9	AST Validator	Parses SQL AST via sqlglot — blocks non-SELECT statements
10	EXPLAIN Check	Runs MySQL EXPLAIN to validate query cost and structure
11	DB Executor	Executes query via SQLAlchemy connection pool, fetches up to 500 rows
12	Self-Correction	On any failure, feeds error back to LLM for up to 3 retries
13	Report Generator	Generates CSV, Excel, PDF report files saved to /reports/
14	Summary Generator	Creates natural language summary of results
15	Response JSON	Returns standardized result object to frontend
16	React Rendering	Displays SQL, table, chart, summary, and download buttons

3. Technology Stack

Backend

Technology	Version	Purpose
Python	3.10+	Primary backend language
FastAPI	0.141.1	High-performance REST API framework with async support
Uvicorn	0.52.1	ASGI server for production-grade HTTP handling
SQLAlchemy	2.0.51	ORM and connection pooling for MySQL
PyMySQL	1.2.0	MySQL driver for Python
Ollama	0.6.2	Local LLM server client (qwen2.5-coder:7b model)
ChromaDB	1.5.9	In-process vector database for semantic schema search
Sentence-Transformers	5.6.1	Embedding model (all-MiniLM-L6-v2) for RAG
sqlglot	≥30.0.0	SQL AST parser and validator (dialect-aware)
pandas	3.0.5	Data manipulation, serialization, and report generation
openpyxl	≥3.1.0	Excel report generation (.xlsx)
ReportLab	≥5.0.0	PDF report generation
python-dotenv	1.2.2	Environment variable management via .env file
Pydantic	2.13.4	Request/response validation and data modeling
tenacity	9.1.4	Retry logic for resilient LLM calls
rich	15.0.0	Enhanced console output and logging

Frontend

Technology	Version	Purpose
React	19.2.8	UI component library
Vite	8.2.0	Ultra-fast build tool and dev server
Recharts	3.10.1	Auto-chart visualization of query results
Lucide React	1.28.0	Icon library
Vanilla CSS	—	Custom styling (no framework, full control)

Technology	Version	Purpose
oxlint	1.75.0	Fast JavaScript linter

Infrastructure & Tools

Tool	Purpose
MySQL	Primary relational database (read-only access)
Ollama	Local LLM inference server running qwen2.5-coder:7b
ChromaDB	Vector store for schema embeddings
knowledge/ directory	JSON-based knowledge base for schema, entities, and business terms
reports/ directory	Auto-generated query report files (CSV, Excel, PDF)

4. Project Structure & File Reference

Project Directory Tree

```
Agent/                                ← Project root
├── .env                              ← Database credentials (never commit)
├── requirements.txt                   ← Python dependencies
├── app/
│   ├── main.py                       ← Backend entry point (uvicorn runner)
│   └── api/
│       ├── main.py                   ← FastAPI app, all REST endpoints
│       └── agent/
│           ├── sql_agent.py          ← Core AI agent pipeline orchestrator
│           ├── ambiguity_checker.py ← Query clarity validation
│           ├── query_planner.py      ← Deterministic execution plan builder
│           └── llm/
│               ├── sql_generator.py  ← Ollama LLM + deterministic fallback
│               └── qwen.py           ← Qwen model helper
│           └── prompt/
│               └── prompt_builder.py ← Structured SQL prompt construction
│           └── database/
│               ├── connection.py     ← MySQL connection test script
│               ├── read_executor.py  ← Read-only query executor + EXPLAIN
│               ├── metadata_extractor.py ← Enterprise schema metadata builder
│               ├── schema_extractor.py ← Table/column schema introspection
│               └── allowed_tables.py  ← TARGET_SCOPE_TABLES whitelist
│           └── knowledge/
│               ├── knowledge_graph.py ← BusinessKnowledgeGraph core class
│               ├── db_profiler.py     ← Enum/sample value profiler
│               ├── knowledge_builder.py ← Orchestrates knowledge base build
│               ├── relationship_builder.py ← FK and inferred relationship mapping
│               └── schema_builder.py   ← Schema JSON builder
│           └── embedding/
│               ├── chroma_builder.py  ← ChromaDB vector store builder
│               ├── embedding_builder.py ← Sentence transformer embedding wrapper
│               └── sql_history_embedding.py ← Historical SQL embedding
```

- ■■■ retriever/
- ■ ■■■ retriever.py ← Schema/history retrieval functions
- ■■■ validator/
- ■ ■■■ sql_ast_validator.py ← sqlglot-based AST SQL validation
- ■ ■■■ sql_optimizer.py ← SQL optimization rules
- ■ ■■■ sql_validator.py ← Rule-based SQL pre-validation
- ■■■ sql_history/
- ■ ■■■ query_history.py ← Query history CRUD (JSON file-based)
- ■■■ utils/
- ■ ■■■ report_generator.py ← CSV/Excel/PDF report generators
- ■ ■■■ sql_cleaner.py ← Raw LLM output SQL sanitizer
- ■ ■■■ summary_generator.py ← Natural language result summary
- ■■■ static/ ← Built frontend assets (Vite output)
- frontend/
- ■■■ src/
- ■ ■■■ App.jsx ← Root React component + routing
- ■ ■■■ index.css ← Global design system CSS
- ■ ■■■ components/
- ■ ■■■ CenterChat.jsx ← Main chat interface
- ■ ■■■ DataGrid.jsx ← Results table renderer
- ■ ■■■ AutoChart.jsx ← Auto-charting with Recharts
- ■ ■■■ DatabaseSchemaPanel.jsx ← Right panel schema browser
- ■ ■■■ HistoryView.jsx ← Query history & audit tab
- ■ ■■■ AdminPanel.jsx ← Admin dashboard
- ■ ■■■ Settings.jsx ← Settings configuration panel
- ■ ■■■ Sidebar.jsx ← Navigation sidebar
- ■■■ package.json ← Node.js dependencies
- ■■■ vite.config.js ← Vite configuration + API proxy
- knowledge/ ← Auto-generated knowledge base JSON files
- ■■■ schema/ ← schema_metadata.json, enum_dictionary.json
- ■■■ graph/ ← entity/column/business dictionaries
- ■■■ patterns/ ← query_patterns.json (learned patterns)
- ■■■ history/ ← execution_history.json
- reports/ ← Auto-generated query reports
- logs/ ← Application logs

5. Backend — Python / FastAPI

5.1 API Endpoints Reference

The backend exposes a REST API on **http://localhost:8000**. All endpoints return JSON.

Method	Endpoint	Description	Auth
GET	/	Serve React frontend SPA	None
GET	/health	System health check (DB + LLM status)	None
GET	/schema	Return {table: [columns]} schema map	None
POST	/query	Main NL→SQL query endpoint (synchronous)	None
POST	/query/stream	Streaming SSE version with step events	None
POST	/export/csv	Export result data as CSV file	None
POST	/export/excel	Export result data as Excel (.xlsx)	None
POST	/export/pdf	Export result data as PDF	None
POST	/export/json	Export result data as JSON file	None
POST	/export/sql	Download the generated SQL as .sql file	None
GET	/history	Fetch query history (limit, search params)	None
POST	/history/favorite	Toggle favorite on a history entry	None
POST	/history/delete	Delete a history entry by timestamp	None
GET	/admin/tables	List all tracked tables with row counts	None
GET	/admin/schema	Full raw schema metadata JSON	None
GET	/admin/knowledge-graph	Knowledge graph nodes and edges	None
GET	/admin/status	Comprehensive system status	None
GET	/admin/stats	Query analytics stats (latency, confidence)	None
GET	/admin/settings	Get current LLM/embedding settings	None
POST	/admin/settings	Update LLM model, temperature, top_k, etc.	None
POST	/admin/rebuild-embeddings	Rebuild ChromaDB vector embeddings	None
POST	/admin/refresh-metadata	Re-extract database schema metadata	None

5.1.1 POST /query — Request & Response

The primary endpoint used by the chat interface:

```
# POST /query Request
```

```
# Request Body
```

```
{
  "question": "Show all wallet transactions for July 2026",
  "execute": true
}
```

```
# POST /query Response
```

```
# Successful Response
```

```
{
  "status": "success",
  "question": "Show all wallet transactions for July 2026",
  "sql_query": "SELECT id, user_id, amount, ... FROM wallet_transaction WHERE ...",
  "execution_time": 128.5,
  "rows_returned": 47,
  "summary": "There were 47 transactions in July 2026 with total █1,23,450.",
  "results": [{...}, ...],
  "columns": ["id", "user_id", "amount", "transaction_type", "created_at"],
  "report_urls": {"csv": "/reports/abc12345.csv", "excel": "...", "pdf": "..."},
  "generated_sql": "...",
  "explanation": "...",
  "confidence_score": 95,
  "thinking_steps": ["Step 1: ...", "Step 2: ..."],
  "affected_tables": ["wallet_transaction"],
  "benchmarks": {"total_ms": 4200, "llm_generation_ms": 3100, ...}
}
```

5.1.2 Ambiguous Query Response

```
# Ambiguous Query Response
```

```
{
  "status": "ambiguous",
  "clarification": "Your prompt is underspecified. Please clarify:",
}
```

```

"options": [
    "Today's Transactions",
    "Last 7 days Transactions",
    "Show Wallet Transactions"
]
}

```

5.2 AI Agent Pipeline (app/agent/sql_agent.py)

The **run_agent()** function is the core orchestrator. It implements a multi-step reasoning pipeline with up to 3 self-correction retries:

Step	Function	Description
1	check_ambiguity()	Validates query clarity against DB enum values and term vocabulary
2	kg.resolve_business_query()	Detects relevant tables, joins, date ranges from the Knowledge Graph
3	create_plan()	Builds a structured execution plan dict with tables, columns, joins, terms
4	build_sql_prompt()	Constructs a tight, structured LLM prompt from the execution plan
5	generate_sql()	Calls Ollama LLM or deterministic synthesizer; extracts SQL from response
6	clean_sql()	Strips markdown code fences, truncates at semicolon
7	validate_sql()	AST validation via sqlglot — blocks non-SELECT, checks table/column names
8	get_execution_plan()	Runs EXPLAIN FORMAT=JSON to validate and estimate query cost
9	execute_read_query()	Executes SQL via connection pool; fetches max 500 rows
10	compute_confidence()	Calculates 0-100 confidence score from validation and plan data
11	generate_natural_summary()	Creates human-readable results summary with totals/counts
12	save_reports_to_disk()	Generates and saves CSV/Excel/PDF report files
13	log_query_history()	Persists query metadata to history JSON store

5.3 Knowledge Graph (app/knowledge/knowledge_graph.py)

The **BusinessKnowledgeGraph** is a singleton in-memory graph built from JSON metadata files at startup. It provides:

Table Node Registry	All target tables registered as graph nodes with metadata.
Explicit FK Edges	Foreign key constraints from DB schema stored as directed edges.
Inferred FK Edges	Columns ending in _id infer relationships via column dictionary.

BFS Join Resolution	Finds shortest join path between any two tables using BFS traversal.
Alias Resolution	60+ business term aliases mapped to target tables (e.g. 'retailer' → users).
Date Range Parsing	Extracts 'July 2026', 'last month', 'this month' into start/end date filters.
Enum Value Mapping	DB enum values loaded for ambiguity checking and business term filters.
Pattern Learning	Successful queries are saved as learned patterns to improve future generation.

Knowledge base JSON files loaded at startup:

File	Contents
knowledge/schema/schema_metadata.json	Full table/column metadata extracted from MySQL
knowledge/schema/enum_dictionary.json	Enum/distinct values for categorical columns
knowledge/schema/sample_values.json	Sample values for context-aware prompting
knowledge/graph/entity_dictionary.json	Entity-to-table mappings
knowledge/graph/column_dictionary.json	Column-to-table cross-reference
knowledge/graph/business_dictionary.json	Business terminology with SQL filter conditions
knowledge/graph/join_graph.json	Computed join adjacency list
knowledge/graph/relationship_metadata.json	All FK and inferred edges
knowledge/patterns/query_patterns.json	Learned successful query patterns
knowledge/history/execution_history.json	Historical execution records

5.4 SQL Generator & LLM Integration (app/llm/sql_generator.py)

The generator first checks if **Ollama is online** (HTTP ping to port 11434 with 0.5s timeout). If online, it calls the **qwen2.5-coder:7b** model with strict options. If offline, it instantly falls back to a **deterministic rule-based SQL synthesizer**.

Parameter	Value	Description
Model	qwen2.5-coder:7b	Default local LLM model (configurable via settings)
temperature	0.0	Deterministic output, no randomness
top_p	0.9	Nucleus sampling threshold

Parameter	Value	Description
num_predict	200	Max token output limit
stop tokens	[``\n\n', 'Explanation:']	Stop generation at these tokens
keep_alive	60m	Keep model loaded in memory for 60 minutes
system role	Expert MySQL Analyst	System message constraining the model to SQL-only output

5.4.1 Deterministic SQL Synthesizer (Fallback)

When Ollama is offline, the synthesizer parses the structured execution plan prompt using regex to extract tables, columns, joins, filters, and date ranges — then builds a valid SQL query programmatically without any LLM involvement. This ensures **100% uptime** even without a running LLM server.

5.5 Prompt Engineering (app/prompt/prompt_builder.py)

The prompt builder converts a structured execution plan into a tightly constrained LLM prompt:

```
# LLM Prompt Template

You are a Senior MySQL Enterprise Data Analyst for JGH.
Translate this business question into a valid MySQL SELECT query.

Business Question:
{intent}

Available Tables & Allowed Columns:
Table `users` with columns: [id, name, email, mobile_number, ...]
Table `role` with columns: [id, name, created_at]

Business Term Mappings:
- 'retailer': use filter `users.user_role = 2` (User role 2 means retailer)

Required Joins:
users.user_role = role.id

CRITICAL RULES:
1. ONLY SELECT columns that exist in the Allowed Columns list above.
2. If business term mappings are provided, use their exact filter.
3. NEVER invent imaginary columns.
4. ALWAYS apply LIMIT 500 unless calculating aggregates.
5. Output ONLY the query in ```sql ``` codeblock.
```

5.6 SQL AST Validator (app/validator/sql_ast_validator.py)

Uses **sqlglot** to parse SQL into an Abstract Syntax Tree. Validation steps:

- Root statement **MUST** be SELECT (no INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, etc.)
- Forbidden node types are scanned: Insert, Update, Delete, Drop, Alter, Create, Commit, Rollback, Execute, Command
- All referenced tables must be in the TARGET_SCOPE_TABLES allowlist
- All referenced columns must exist in the schema metadata for those tables
- Any validation failure raises SQLValidationError — triggering a retry with error feedback

5.7 Database Layer (app/database/)

File	Purpose
read_executor.py	execute_read_query(): SQLAlchemy connection pool, EXPLAIN support, Decimal/datetime
metadata_extractor.py	generate_enterprise_knowledge_base(): Deep schema introspection with column types, F
schema_extractor.py	Lightweight schema extraction helpers
allowed_tables.py	TARGET_SCOPE_TABLES: whitelist of 8 target enterprise tables
connection.py	Standalone MySQL connection test and database lister

Connection pool configuration:

```
# SQLAlchemy Pool Config
engine = create_engine(
    db_url,
    pool_size=10,      # Base pool connections
    max_overflow=20,    # Extra connections on demand
    pool_recycle=3600,  # Recycle connections after 1 hour
    pool_pre_ping=True  # Test connections before use
)
```

5.8 Embedding & Vector Store (app/embedding/)

ChromaDB is used as an in-process vector database. Schema descriptions are embedded using **all-MiniLM-L6-v2** (Sentence Transformers) and stored as vector embeddings. The retriever performs cosine similarity search to find the most relevant schema context for a given query — enabling RAG (Retrieval-Augmented Generation) for schema lookups.

File	Purpose
chroma_builder.py	Builds and persists ChromaDB vector store from schema metadata
embedding_builder.py	Sentence transformer embedding wrapper (all-MiniLM-L6-v2)

File	Purpose
sql_history_embedding.py	Embeds historical SQL for similar-query retrieval

5.9 Query History & Audit (app/sql_history/query_history.py)

Every query execution is logged to a JSON file. The history record includes: question, generated_sql, optimized_sql, status (success/error/ambiguous/blocked), execution_time_ms, row_count, affected_tables, error message, confidence_score, execution_plan, explanation, and thinking_steps. History supports filtering by search term and toggling favorites.

5.10 Report Generator (app/utils/report_generator.py)

The report generator creates downloadable output files from query results. All reports are saved to the **reports/** directory with a UUID-based filename:

Format	Library	Details
CSV	Python csv module (streaming)	Streaming response for large datasets; UTF-8 BOM for Excel compatibility
Excel	openpyxl	Styled header row, auto-column widths, worksheet named 'Query Results'
PDF	ReportLab	Branded header, auto-wrapped table, styled column headers, page breaks
JSON	json.dumps	Pretty-printed JSON array of result rows

6. Frontend — React / Vite

The frontend is a **React 19 single-page application** built with Vite 8. It features a dark-themed, glassmorphic design system with smooth animations. The app communicates with the FastAPI backend via HTTP REST calls. In development mode, Vite proxies API calls to localhost:8000.

6.1 Application Screens

Screen	Route/Tab	Description
Query Interface (CenterChat)	chat	Main conversational AI chat interface with SQL display, data table, charts, and controls
History & Audit	history	Full query history with search, re-run, favorite, delete, and SQL preview
Schema Admin	admin	Admin dashboard: system status, table list, knowledge graph viewer, rebuild controls
Settings	settings	Configure LLM model, embedding model, temperature, top_k, max_rows at runtime

6.2 Component Reference

Component File	Description
App.jsx	Root component: sidebar navigation, tab routing, message state management, API call handler
CenterChat.jsx	Full chat interface: message list, SQL code block, execution status card, download button
DataGrid.jsx	Paginated data table with column sorting, overflow handling, and sticky headers
AutoChart.jsx	Recharts-based auto-charting: detects numeric/categorical columns and renders bar/line/area
DatabaseSchemaPanel.jsx	Right panel: live schema browser showing tables and their columns fetched from /schema
HistoryView.jsx	Query history list with search, re-run, favorite toggle, delete, and expandable SQL/result preview
AdminPanel.jsx	Admin dashboard: system stats, table browser, knowledge graph, rebuild embeddings, retrain
Settings.jsx	Full settings form: model name, embedding model, temperature slider, top_k, max_rows
Sidebar.jsx	Navigation sidebar: logo, 'New Query' button, nav items, backend status indicator

6.3 Design System

The application uses a custom CSS design system defined in **index.css** and component-level CSS files. Key design tokens:

Token	Value	Usage
--bg-primary	#0F172A	Main background (dark navy)

Token	Value	Usage
--bg-surface	#1E293B	Card/panel surface
--accent-indigo	#6366F1	Primary brand color
--accent-cyan	#22D3EE	Secondary accent
--text-primary	#F1F5F9	Primary text color
--text-muted	#64748B	Muted/secondary text
Font	System UI / Segoe UI	System font stack for performance

7. Environment Configuration (.env)

The application is configured via a `.env` file in the project root. This file must NEVER be committed to version control (it is in `.gitignore`).

[illegible]

Variable	Required	Default	Description
DB_HOST	Yes	—	MySQL server hostname or IP address
DB_PORT	Yes	3306	MySQL server port number
DB_NAME	Yes	—	Target database name
DB_USER	Yes	—	MySQL username with SELECT privileges
DB_PASSWORD	Yes	—	MySQL user password
LLM_MODEL	No	qwen2.5-coder:7b	Ollama model to use for SQL generation

■ **SECURITY:** The database user must have READ-ONLY access (SELECT only). Never use a user with write permissions. The .env file must never be committed to Git.

8. Installation & Setup Guide

8.1 Prerequisites

Requirement	Version	Notes
Python	3.10+	Required for the backend
Node.js	18+	Required for the frontend build
npm	9+	Node package manager
MySQL	5.7+ or 8.0+	Target database server (read access required)
Ollama	Latest	Optional — provides local LLM inference (install from ollama.ai)
Git	Any	For cloning the repository

8.2 Backend Setup (Step-by-Step)

Step 1: Clone the repository

```
git clone <repository-url>
cd Agent
```

Step 2: Create Python virtual environment

```
python -m venv venv
# Windows:
venv\Scripts\activate
# Linux/Mac:
source venv/bin/activate
```

Step 3: Install Python dependencies

```
pip install -r requirements.txt

# NOTE: This installs ~107 packages including:
# FastAPI, SQLAlchemy, ChromaDB, Sentence-Transformers,
# Ollama client, pandas, ReportLab, openpyxl, sqlglot, etc.
```

Step 4: Configure environment variables

```
# Create .env file in project root:
DB_HOST=your_mysql_host
DB_PORT=3306
DB_NAME=your_database_name
DB_USER=readonly_user
DB_PASSWORD=your_password
```

Step 5: Verify database connection

```
python app/database/connection.py
# Should print: ■ Connected Successfully to MySQL Server!
```

Step 6: Generate knowledge base (first run only)

```
# This runs automatically at startup, or manually:
python -c "from app.database.metadata_extractor import generate_enterprise_knowledge_base; generate_enterprise_knowledge_base()"
```

8.3 Frontend Setup

Step 1: Navigate to frontend directory

```
cd frontend
```

Step 2: Install Node.js dependencies

```
npm install
# Installs: React 19, Vite 8, Recharts, Lucide React, oxlint
```

Step 3: Verify Vite config proxy

```
# vite.config.js proxies /query, /schema, /history,
# /admin, /export, /reports, /health → http://127.0.0.1:8000
```

8.4 Ollama LLM Setup (Optional but Recommended)

```
# Ollama Setup
# 1. Download Ollama from https://ollama.ai
#    and install for your OS (Windows/Mac/Linux)
# 2. Pull the qwen2.5-coder model (~4GB download):
```

```
ollama pull qwen2.5-coder:7b

# 3. Verify it works:
ollama run qwen2.5-coder:7b

# 4. Ollama runs as a background service on port 11434.
#   The app auto-detects Ollama availability.
#   If offline, the deterministic SQL synthesizer is used.
```

9. Running the Application

9.1 Start the Backend Server

```
# Backend Startup

# Navigate to project root
cd Agent

# Activate virtual environment
venv\Scripts\activate      # Windows
source venv/bin/activate    # Linux/Mac

# Start FastAPI backend (auto-reload mode for development)
python app/main.py

# Or directly with uvicorn:
uvicorn app.api.main:app --host 0.0.0.0 --port 8000 --reload

# Expected startup output:
[STARTUP] Pre-loading Knowledge Graph...
[STARTUP COMPLETE] Enterprise AI SQL Agent ready on http://localhost:8000

# API docs available at:
# http://localhost:8000/docs      (Swagger UI)
# http://localhost:8000/redoc    (ReDoc)
```

9.2 Start the Frontend Dev Server

```
# Frontend Dev Server

# In a NEW terminal, navigate to frontend:
cd Agent/frontend

# Start Vite development server:
npm run dev

# Expected output:
VITE v8.2.0 ready in 350 ms
→ Local:   http://localhost:3000/
→ Network: http://0.0.0.0:3000/

# Open browser to: http://localhost:3000
```

9.3 Build & Serve Frontend (Production / Embedded Mode)

```
# Production Build & Serve

# Build the frontend (outputs to app/static/)

cd Agent/frontend

npm run build

# The FastAPI backend serves the built files at root /

# So you only need to run:

python app/main.py

# And access the full app at:

# http://localhost:8000

# This is the PRODUCTION deployment mode –

# single process, no separate frontend server needed.
```

9.4 Health Check

```
# Health Check

# Verify the backend is running:

curl http://localhost:8000/health

# Expected response:

{
  "status": "online",
  "database_connected": true,
  "target_tables_count": 8,
  "llm_model": "qwen2.5-coder:7b"
}
```

9.5 Rebuild Vector Embeddings (after schema changes)

```
# Rebuild Embeddings

# Via API:

curl -X POST http://localhost:8000/admin/rebuild-embeddings

# Via Admin UI:

# Navigate to Schema Admin tab → click 'Rebuild Embeddings'

# Refresh schema metadata (after adding new tables):
```

```
curl -X POST http://localhost:8000/admin/refresh-metadata
```

10. API Usage Examples

Query: Show all retailers

```
curl -X POST http://localhost:8000/query \  
  -H 'Content-Type: application/json' \  
  -d '{"question": "Show all retailers", "execute": true}'
```

Query: Wallet transactions for July 2026

```
curl -X POST http://localhost:8000/query \  
  -H 'Content-Type: application/json' \  
  -d '{"question": "Show wallet transactions for July 2026"}'
```

Query: Count of users by role

```
curl -X POST http://localhost:8000/query \  
  -H 'Content-Type: application/json' \  
  -d '{"question": "How many users are there by role?"}'
```

Export results as CSV

```
curl -X POST http://localhost:8000/export/csv \  
  -H 'Content-Type: application/json' \  
  -d '{"columns": ["id", "name"], "data": [{...}], "filename": "users_report"}'
```

Fetch query history (last 50)

```
curl 'http://localhost:8000/history?limit=50'
```

Rebuild schema embeddings

```
curl -X POST http://localhost:8000/admin/rebuild-embeddings
```

Update LLM settings

```
curl -X POST http://localhost:8000/admin/settings \  
  -H 'Content-Type: application/json' \  
  -d '{...}'
```

```
-d '{"model_name": "qwen2.5-coder:14b", "temperature": 0.1, "top_k": 6}'
```

Get system status

```
curl http://localhost:8000/admin/status
```

11. Data Flow Walkthrough

This section traces a complete end-to-end example: **"Show me all withdrawal requests in July 2026"**

1. User Input	User types 'Show me all withdrawal requests in July 2026' in the chat and presses Enter.
2. Frontend POST	React calls POST /query with {question: '...', execute: true}.
3. Ambiguity Check	check_ambiguity() scans enum values — 'withdrawal requests' matches known DB terms → NOT ambiguous → continues.
4. Knowledge Graph Resolution	resolve_business_query() maps 'withdrawal' → withdrawal_request table. 'July 2026' → date range 2026-07-01 to 2026-08-01.
5. Execution Plan	create_plan() returns: {intent: '...', tables: ['withdrawal_request'], table_columns: {withdrawal_request: [id, user_id, amount, ...]}, date_filters: {start: '2026-07-01', end: '2026-08-01'}, relationships_required: []}.
6. Prompt Building	build_sql_prompt() creates a structured prompt with table columns, date range, and 5 CRITICAL RULES.
7. LLM Generation	generate_sql() calls Ollama qwen2.5-coder:7b. Response includes SQL in ```sql blocks. SQL is extracted and cleaned.
8. AST Validation (Attempt 1)	validate_sql() parses the SQL with sqlglot. Confirms SELECT statement, checks withdrawal_request is in allowed tables, verifies all columns exist.
9. EXPLAIN Check	get_execution_plan() runs EXPLAIN FORMAT=JSON — query is valid, cost estimated at 45.2 → continues.
10. DB Execution	execute_read_query() runs SQL via SQLAlchemy pool. Returns 23 rows in 85ms.
11. Report Generation	save_reports_to_disk() saves report_abc12345.csv, .xlsx, .pdf to /reports/
12. Summary	generate_natural_summary() sees 'amount' column, sums to █4,56,200 → 'There were 23 withdrawal requests in July 2026 with total █4,56,200.'
13. History Log	log_query_history() saves full record including SQL, execution time, confidence=100, affected_tables=['withdrawal_request'].

14. Response JSON	FastAPI returns status='success', sql_query, results, columns, summary, report_urls, benchmarks.
15. Frontend Render	React renders: AI message with summary, SQL block (copy button), ExecutionStatus card (85ms, 23 rows), DataGrid (paginated table), AutoChart (bar chart), download buttons (CSV/Excel/PDF).

12. Security & Read-Only Enforcement

Agentic Analyst enforces strict read-only access at multiple independent layers:

Layer 1: Database User Permissions

The MySQL user configured in `.env` should have SELECT-only privileges. No INSERT, UPDATE, DELETE, DROP, or DDL permissions should be granted.

Layer 2: AST-Level SQL Validation

Every generated SQL is parsed into an AST by `sqlglot` before execution. Non-SELECT root nodes immediately raise `SQLValidationError`, triggering a retry. Forbidden node types: Insert, Update, Delete, Drop, Alter, Create, Commit, Rollback, Execute, Command.

Layer 3: Table Allowlist

`TARGET_SCOPE_TABLES` in `allowed_tables.py` defines exactly which tables can be queried. Any SQL referencing tables outside this list is rejected.

Layer 4: Column Allowlist

The AST validator cross-references every column reference against the schema metadata. Columns that don't exist in the target table are rejected.

Layer 5: EXPLAIN Validation

MySQL `EXPLAIN FORMAT=JSON` validates the query structure before execution. Invalid queries that pass AST checks are caught here.

Layer 6: Prompt Constraints

The LLM system prompt explicitly instructs the model to output only SELECT statements. The prompt is structured to minimize SQL injection risks.

Layer 7: SQL Cleaner

The SQL cleaner strips markdown, truncates after the first semicolon, and sanitizes the raw LLM output before any validation.

Layer 8: Row Limits

All queries are subject to LIMIT 500 (configurable via settings). Aggregate queries are exempt from LIMIT but are COUNT/SUM only.

■ **SECURITY SUMMARY:** A malicious or incorrect LLM output would need to bypass all 8 independent security layers to cause any data modification. Each layer independently blocks non-SELECT operations.

13. Troubleshooting & FAQ

■ Backend won't start

Solution: Ensure the virtual environment is activated. Run: `venv\Scripts\activate` (Windows). Check that all dependencies are installed: `pip install -r requirements.txt`

■ Database connection failed

Solution: Verify `.env` variables are correct. Test directly: `python app/database/connection.py`. Ensure MySQL server is running and the user has `SELECT` access to `DB_NAME`.

■ LLM not generating SQL

Solution: Check if Ollama is running: `curl http://localhost:11434/api/tags`. If offline, the deterministic synthesizer will be used automatically. Pull the model: `ollama pull qwen2.5-coder:7b`

■ 'Table X not allowed' validation error

Solution: The table is not in `TARGET_SCOPE_TABLES` (`allowed_tables.py`). Add the table name to the allowlist and restart the backend.

■ 'Column X does not exist' error

Solution: The schema metadata may be stale. Run: `POST /admin/refresh-metadata` to re-extract the latest schema from the database.

■ Frontend shows 'Failed to connect to backend'

Solution: Ensure the backend is running on port 8000. The frontend dev server proxies API calls to `127.0.0.1:8000`.

■ ChromaDB / embedding errors on startup

Solution: Run: `POST /admin/rebuild-embeddings` from the Admin panel or via curl. Delete the `app/chroma` directory and restart if corruption is suspected.

■ Slow SQL generation

Solution: The LLM model is loading. After the first query, the model is kept in memory for 60 minutes (`keep_alive=60m`). Subsequent queries are fast (~1-3s).

■ Knowledge base JSON files missing

Solution: Start the backend — knowledge base is auto-generated on first startup. Or manually: from `app.database.metadata_extractor` import `generate_enterprise_knowledge_base`; `generate_enterprise_knowledge_base()`

■ Reports directory not found

Solution: The `/reports` directory is auto-created by the API on startup. Ensure the process has write permissions to the project root.

■ Ambiguous query keeps firing

Solution: Edit `knowledge/business_metadata.json` to add or refine business terms. Run `POST /admin/refresh-metadata` to reload.

14. Glossary

Term	Definition
AI Agent	Autonomous software that perceives input, reasons about it, and takes actions to achieve goals
AST (Abstract Syntax Tree)	A tree representation of the grammatical structure of SQL used for validation
ChromaDB	Open-source, in-process vector database for storing and searching embeddings
EXPLAIN	MySQL command that shows the query execution plan without running the query
FastAPI	Modern Python web framework for building APIs with automatic validation and docs
Hallucination	When an LLM generates incorrect or fabricated information (e.g., non-existent columns)
Knowledge Graph	An in-memory network of entities (tables, columns) and their relationships
LLM (Large Language Model)	AI model trained on text data; here used as qwen2.5-coder:7b via Ollama
NL2SQL	Natural Language to SQL — converting English questions to database queries
Ollama	Local LLM server that runs models on your machine without internet dependency
Prompt Engineering	The craft of structuring LLM input to guide desired output
RAG (Retrieval-Augmented Generation)	Enhancing LLM prompts with retrieved contextual data (schema embeddings)
Self-Correction	The agent's ability to detect errors and retry with error feedback (up to 3 times)
Sentence Transformers	ML models that convert text into semantic vector embeddings for similarity search
sqlglot	Python library for parsing, analyzing, and transforming SQL across dialects
SQLAlchemy	Python SQL toolkit and ORM with connection pooling
SSE (Server-Sent Events)	HTTP streaming protocol used by /query/stream endpoint
TARGET_SCOPE_TABLES	The allowlist of 8 enterprise tables that the agent is permitted to query
Vector Embedding	A numerical vector representation of text capturing semantic meaning
Vite	Next-generation frontend build tool and dev server used for the React app